DevOps Lab Assessment - Experiment 6

Docker Storage: Ephemeral Container Layers, Named Volumes & Bind Mounts

COURSE / LAB:	DevOps Lab 2 (Containerization & K8s)	**ASSESSMENT:**	Question 6 (5 Marks)
TOPIC:	Docker Storage & Data Persistence	**ENVIRONMENT:**	Docker 29 / Minikube v1.36 / kubectl

1. Experiment Objective & Problem Statement

By default, all files created inside a container are stored on a thin, ephemeral writable layer. The objective of this experiment is to empirically demonstrate: (1) data loss occurring upon container deletion in the default storage driver, (2) permanent persistence across container lifecycles using **Docker Named Volumes**, and (3) bidirectional host-container synchronization using **Docker Bind Mounts**.

2. Storage Drivers & The Copy-on-Write (CoW) Paradigm

Docker utilizes union filesystems (OverlayFS). An image is comprised of immutable, read-only layers. When a container is instantiated, Docker adds a thin read-write **Container Layer** on top. Any modified or newly created file resides exclusively in this container layer. When the container is destroyed (docker rm), this layer is immediately deleted from the host disk. To persist data, Docker provides **Named Volumes** (managed in /var/lib/docker/volumes) and **Bind Mounts** (direct mapping to arbitrary host directories).

3. Experiment 6.1: Demonstration of Data Loss in Ephemeral Container Layer

```
$ # 1. Launch ephemeral container without external volume mounts docker run -d --name temp-worker alpine:latest sleep 3600 # 2. Write critical state data to container layer docker exec temp-worker sh -c "echo 'SESSION_TOKEN_99812_DATA' > /tmp/state.txt" docker exec temp-worker cat /tmp/state.txt # 3. Delete the container docker rm -f temp-worker # 4. Launch new container from identical image docker run -d --name temp-worker-2 alpine:latest sleep 3600 docker exec temp-worker-2 cat /tmp/state.txt
temp-worker (running) SESSION_TOKEN_99812_DATA temp-worker (deleted) temp-worker-2 (running) cat: can't open '/tmp/state.txt': No such file or directory OBSERVATION: File /tmp/state.txt was completely destroyed with the container layer.
```

4. Experiment 6.2: Permanent Storage Using Docker Named Volume

A Docker Named Volume is created and mounted into vol-producer. After writing persistent data, the container is destroyed. A brand new container (vol-consumer) mounts the same volume and retrieves the data intact.

```
$ # 1. Create Docker named volume docker volume create app-persistent-vol docker volume inspect app-persistent-vol --format "Mountpoint: {{.Mountpoint}}" # 2. Run Producer container and write persistent database file docker run -d --name vol-producer -v app-persistent-vol:/storage alpine sh -c "echo 'PERSISTED_DATABASE_RECORD_V1' > /storage/db.dat" # 3. Forcefully delete Producer container docker rm -f vol-producer # 4. Run Consumer container mounting the same volume and verify data docker run --rm -v app-persistent-vol:/storage alpine cat /storage/db.dat
app-persistent-vol Mountpoint: /var/lib/docker/volumes/app-persistent-vol/_data vol-producer (created and exited 0) vol-producer (deleted) PERSISTED_DATABASE_RECORD_V1 OBSERVATION: Data survived container deletion and was successfully read by a new container!
```

5. Experiment 6.3: Permanent Storage Using Docker Bind Mount

A host directory is mounted into a container. Modifications made by the host are immediately reflected in the container, and modifications made inside the container are preserved on the host after deletion.

```
$ # 1. Create host configuration file mkdir -p ./host_data && echo 'HOST_API_KEY=ABC-987-XYZ' > ./host_data/config.env # 2. Mount host directory into container at /app/config docker run -d --name bind-demo -v $(pwd)/host_data:/app/config alpine sleep 3600 # 3. Read host file from inside container docker exec bind-demo cat /app/config/config.env # 4. Modify file from inside container docker exec bind-demo sh -c "echo 'CONTAINER_APPENDED_TIMESTAMP=2026-11-04' >> /app/config/config.env" # 5. Delete container and verify host file docker rm -f bind-demo && cat ./host_data/config.env
bind-demo (running) HOST_API_KEY=ABC-987-XYZ bind-demo (deleted) HOST_API_KEY=ABC-987-XYZ CONTAINER_APPENDED_TIMESTAMP=2026-11-04 OBSERVATION: Bidirectional sync verified. File permanently stored on host file system.
```

6. Comprehensive Storage Architecture Comparison

Storage Mechanism	Location on Disk	Persistence Lifecycle	Primary Production Use Case
Container Layer	Managed by storage driver	Ephemeral; deleted with container.	Temporary scratch space, build caches.
Named Volume	/var/lib/docker/volumes/...	Permanent; survives container restarts/removals.	Production databases (PostgreSQL, MySQL, Redis).
Bind Mount	User-specified host filesystem	Permanent; decoupled from Docker engine.	Source code mounting for live dev, host config files.

7. Engineering Observations & Conclusion

Key Technical Takeaways

1. **Data Fragility:** The ephemeral container layer must never be trusted for stateful workloads.
2. **Volume Portability:** Docker Named Volumes isolate host filesystem details, making them the standard choice for containerized databases.
3. **Host Coupling:** Bind mounts grant containers direct host path access, requiring strict permission management to prevent unauthorized host file alteration.